

State Space Search

Next level brute force

Key Concept

- Recall the brute force search, we focus on defining the state space and how to enumerate members of the subspace.
- Now, we consider two more aspects of search space
 - Which order should we enumerate the member (BFS and DFS topics)
 - Can we reduce the size of the search space (Backtracking and Branch-and-Bound)
- Introducing the Search Tree (which is just a very close relative of the recursion tree)

Overview

State Space Search

Basics

- Candidate solution
- Set of all candidate solutions
- How to generate
- How to check

State Space Tree Traversing

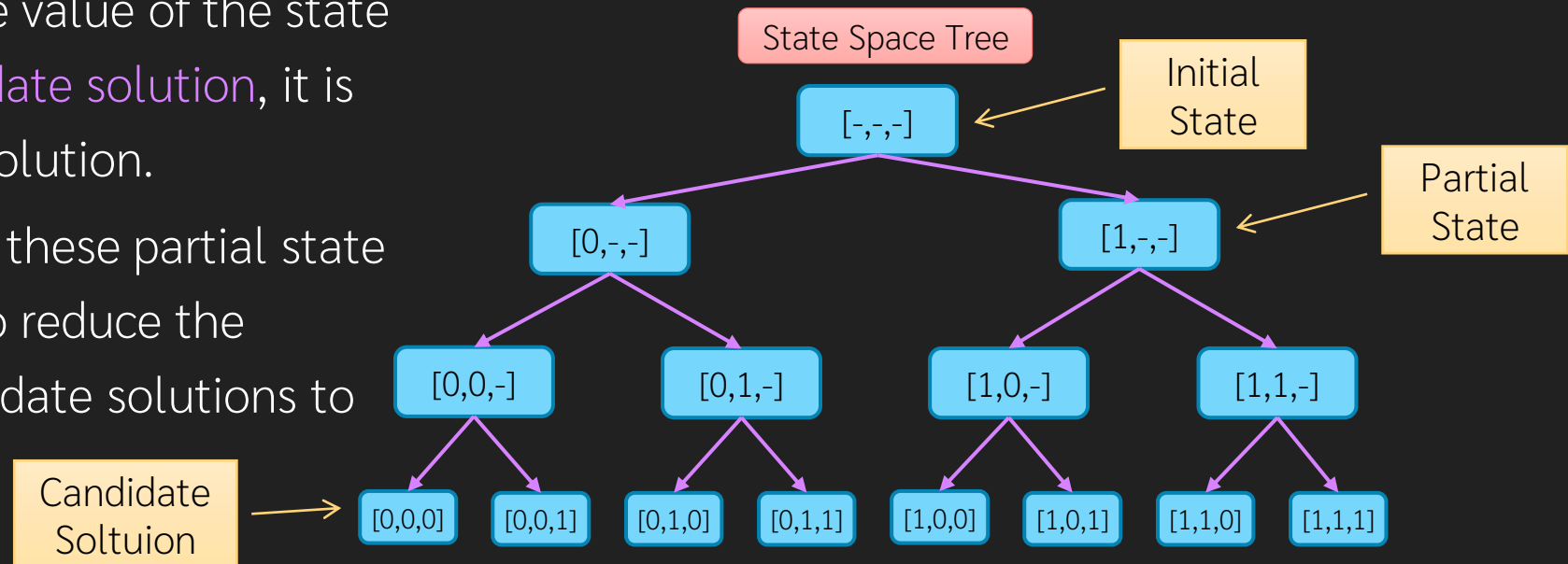
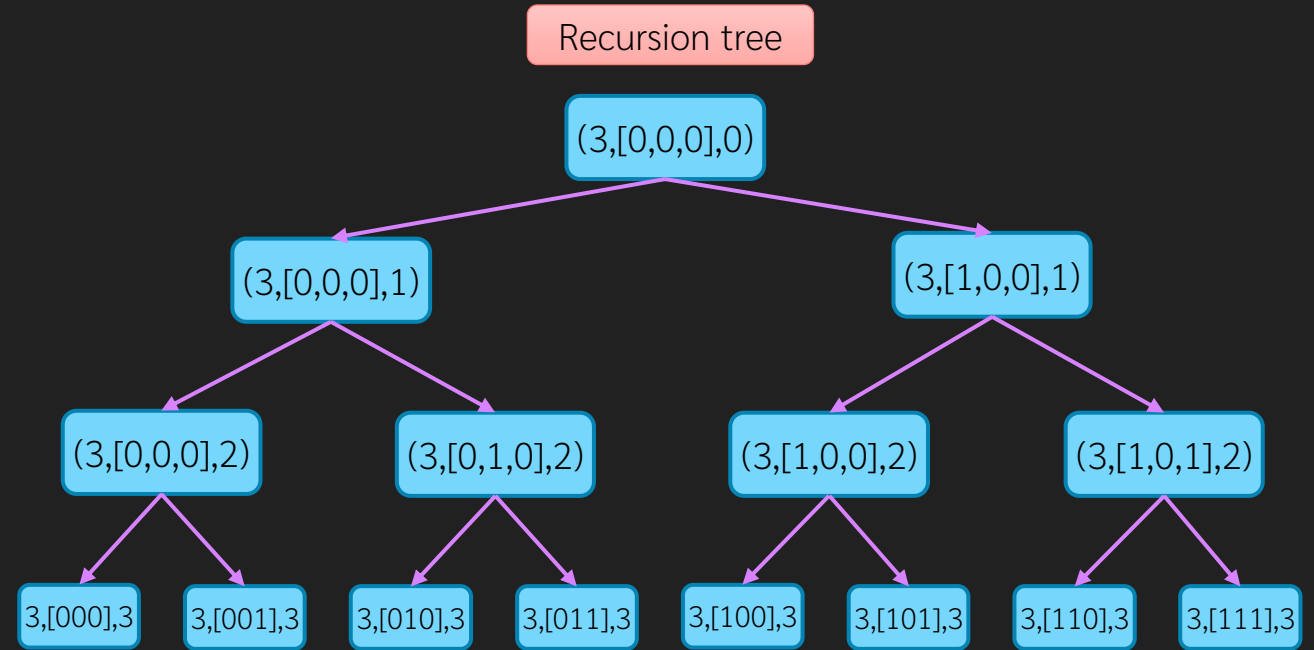
- What is search tree
- Partial solutions
- DFS
- BFS

State Space Tree Optimization

- Backtracking
- Branch and Bound
 - Least-Cost Search

State Space Tree

- A tree that represents **how we generates** each member of the search space
 - Also represent a **partial state**, a state where a solution is being constructed. The value of the state is not the **candidate solution**, it is just a part of a solution.
 - We will consider these partial state and **pruning** it to reduce the number of candidate solutions to be considered.



State type

- Initial state: our starting points
- Partial state: not a full candidate solution but parts of it
- Candidate solution: full solution where it can be solely determined whether it satisfies the constraint

8-Queen Problem

Example on brute force

8-queen problem

- Given a chess board with 8 queens
- A queen is a piece in a chess game that, for one single action, can move as a straight line in all 8-direction (up, down, left, right, all 4 diagonal)
 - If a piece can move in one single action to a cell that has the other piece, we call this a capture
- We must place all queens such that each pairs of queen cannot capture each other

X		X		X
	X	X	X	
X	X		X	X
	X	X	X	
X		X		X

8-queen problem

- The task is to try to place the queens so that they cannot capture
- The board on the right is not a solution because the top left and the bottom right queen can capture each other.

Q							
		Q					
				Q			
						Q	
	Q						
			Q				
					Q		
							Q

8-queen problem

- **Task:** Find all queen placement that is non-capture
- **Input:** null
- **Output:** Every possible placement of 8-queens that does not attach each other
- There is only one instance of this problem

Solving the Problem

- Define the search space
 - What is the search space of this problem?
 - How large it is?
 - Choose an appropriate representation
- Each cells is numbered 1 to 64
- Assume that there is a `check(sol)` function that return true when the placement is correct

1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24
25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56
57	58	59	60	61	62	63	64

Search Space Version 0.1

- Most naïve
 - Combination with replacement
 - There are 8 queens, there are 64 positions for each queens
- Every possible placement of queens
 - Size: 64^8 (approximately 2.8×10^{14})
 - Representation: a set of queen's positions
 - E.g., (1,1) (1,2) (2,5) (4,1) (1,2) (3,4) (8,8) (7,6)
- This includes overlapping placement!!! (two or more queens on the same spot)

V 0.1 implementation

- The most basic 64-bit binary counter
- `check(cell)` has to
 - Check if there is exactly 8 queens
 - Each pair of queens cannot capture each other

```
void e_queen(int step, vector<bool> &cell, int chosen) {  
    if (step <= 64) {  
        cell[step] = true;  
        e_queen(step + 1, cell, chosen);  
        cell[step] = false;  
        e_queen(step + 1, cell, chosen+1);  
    } else {  
        check(cell);  
    }  
}
```

Search Space Version 0.2

- Binary-counter-based search space
 - 64 cells, each cells is either **has**/**not has** a queen
 - Pick exactly 8 cells out of 64
- Try to exclude overlapping by using combination without replacement
- Size: $C_{64,8}$ (approximately 1.7×10^{14})

Version 0.2 implementation

```
void e_queen(int step, vector<bool> &cell, int chosen) {  
    if (step <= 64) {  
        if ((64 - 8) - (step - chosen) > 0) {  
            cell[step] = true;  
            e_queen(step + 1, cell, chosen);  
        }  
        if (8 - chosen > 0) {  
            cell[step] = false;  
            e_queen(step + 1, cell, chosen+1);  
        }  
    } else {  
        check(cell);  
    }  
}
```

Number of remaining
non-queen

Number of
remaining queen

```
e_queen(0, cells, 0);
```

Search Space version 0.3

- Any better way?
- Observation: For each row, there should be only one queen
 - The problem consists of 8 steps
 - Placing each queen on each row
- Size: 8^8
- Representation: sequence of columns
 - E.g., **[1,6,1,7,5,6,7,8]** means that
 - There is a queen at columns 1 on the first row
 - There is a queen at columns 6 on the second row...

Version 0.3 implementation

- Need different version for check function
- Search space size is 8^8
 - approximately $1.6 * 10^7$

```
void e_queen(int step, vector<int> &col) {  
    if (step < 8) {  
        for (int i = 0; i < 8; i++) {  
            col[step] = i;  
            e_queen(step + 1, col);  
        }  
    } else {  
        check(col);  
    }  
}
```


Version 0.4

- Another observation: queens should not be in the same column
 - The solution should never have any column repeated
 - E.g.,
 - (1,2,3,4,5,6,7,1) is bad (column collision)
 - (1,1,3,4,5,6,7,5) is bad as well....
 - (1,2,3,4,5,6,7,8) is good
- Permutation-based
- Size: 40,320

Version 0.4 implementation

```
void e_queen(int step, vector<int> &col, vector<bool> &used) {  
    if (step < 8) {  
        for (int i = 0; i < 8; i++) {  
            if (!used[i]) {  
                used[i] = true;  
                col[step] = i;  
                e_queen(step+1,col,used);  
                used[i] = false;  
            }  
        }  
    } else {  
        check(col);  
    }  
}
```

DFS and BFS Algorithm

Same as one in the graph, but on the state space tree

Why we need to consider the order of states

- Different orders give us different information about the task
 - Some order allow us to prune the state tree better
- There is also a case when a candidate solution can be found as internal node in the search tree, or we might want to find a candidate solution that is closest to the initial state
- Additionally, there is a case where the candidate solution set is infinite

Breadth First Search and Depth First Search

```
storage S
put initial state to S
while S is not empty
    curr = S.remove_one
    if curr is a candidate solution
        check(curr)
    else
        generate all next step from curr
        put them into S
```

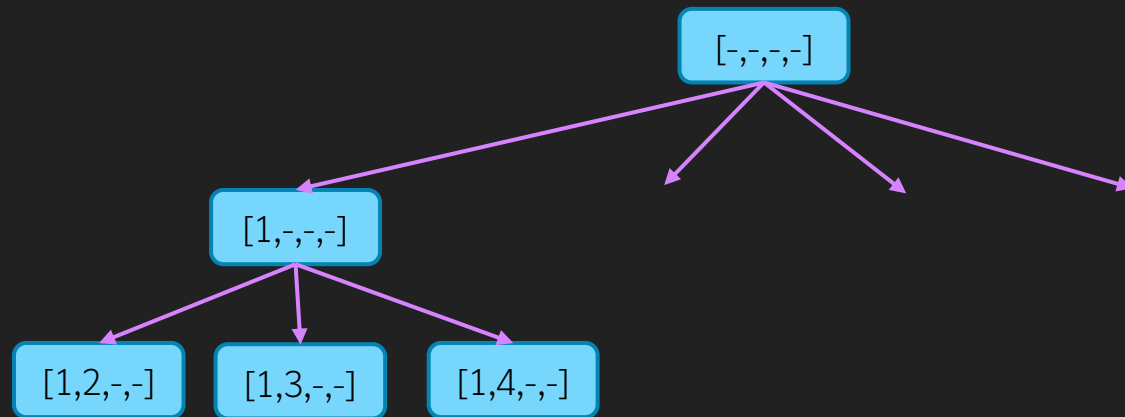
- Search Tree grows according to S
 - and how to generate all next step as well,
- If S is a **stack**, the algorithm is called **DFS**
- If S is a **queue**, the algorithm is called **BFS**

Analogy with the graph search algorithm

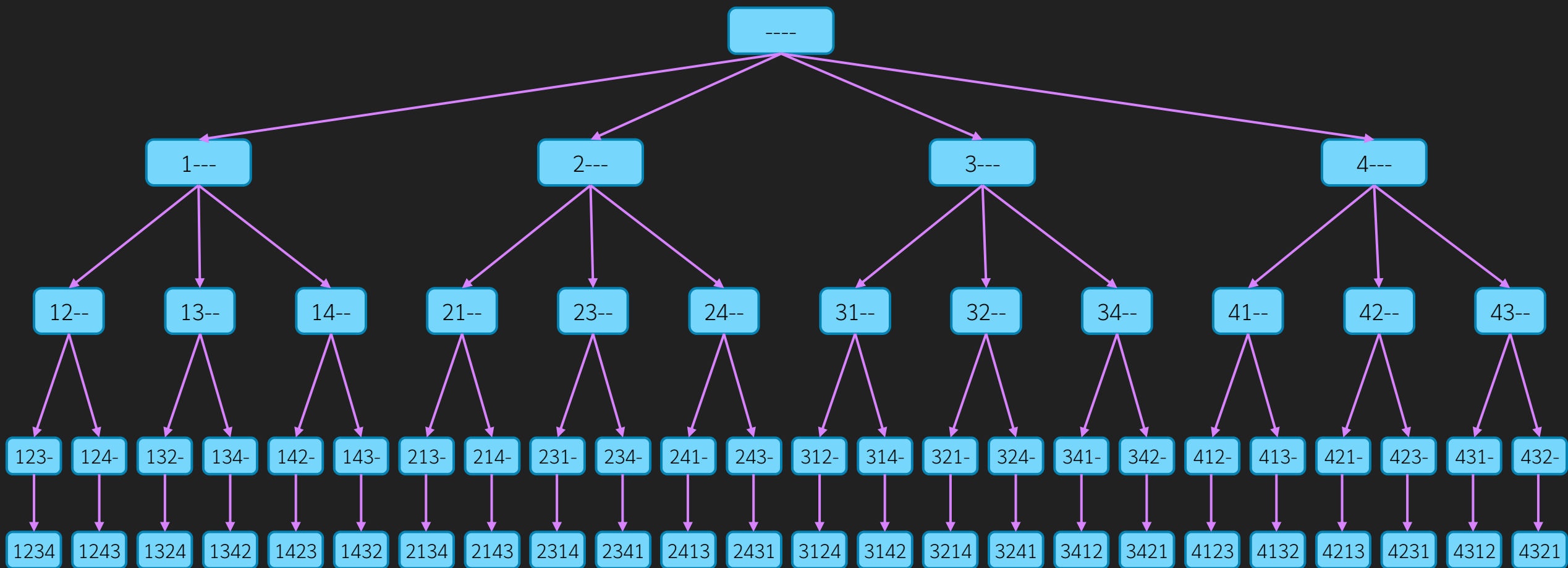
- Notice that these BFS and DFS is the same as the graph search algorithm
- However, the graph is not explicitly given. Node is a state and edge is how we generate the next step from our current state
- The behavior is the same
 - DFS: keep going as long as there is a path
 - BFS: keep exploring based on distance from the root

Example

- Consider 4 queen, using permutation based
- Candidate solution set is permutations of [1,2,3,4]



4 Queen Search Space



Triple and Half Problem

- Input:
 - A number
- Output:
 - A sequence of either $\times 3$ or $/ 2$ operation, start from 1 that is evaluated as the given number
 - Special version: number of operations should be minimal
- Example:
 - Input: 10
 - Output: $1 \times 3 \times 3 \times 3 \times 3 / 2 / 2 / 2$
 - Input: 31
 - Output: $1 \times 3 \times 3 \times 3 \times 3 \times 3 / 2 / 2 / 2 / 2 / 2 \times 3 \times 3 / 2$

Triple and Half Problem

- Search Space
 - Every possible sequence of triple or half
 - Described by a sequence of operation
 - [T,H,T,H,H] where H mean halving ($/2$) and T means tripling ($*3$)
 - [T,H,T,H,H] is " $1*3/2*3/2/2$ "
 - Issue: There is no clear length-limit of sequence!!
 - This is example of infinite candidate solution set
- Generating all solutions
 - In each step, Choose either tripling or halving
- Checking each solution
 - Evaluate the expression
 - and test whether it equals to target

Ver 0.1

```
void DFS(int goal) {
    stack<string> s;
    s.push("1");
    while (!s.isEmpty()) {
        auto curr = s.pop();
        if ( evaluate(curr) == goal) {
            printf("found! %s",curr);
            return ;
        }
        string s1 = curr + "*3";
        string s2 = curr + "/2";
        s.push(s1);
        s.push(s2);
    }
}
```

- Assume that `evaluate(curr)` is a function that calculate a string of numerical expression (e.g., "1*3*3/2*3") into an integer
- Issue
 - Repeat solution?
 - $1 * 3 * 3 / 2 * 3 / 2 \rightarrow 6$
 - $1 * 3 * 3 * 3 / 2 / 2 \rightarrow 6$
 - When will the program stop?
 - Can you draw the search tree?

Version 0.2

- Solve repeated state by using a hash (unordered_set)
- This code on the right does not really solve the repeated state problem
 - Why?

```
void DFS(int goal) {
    stack<string> s;
    unordered_set<string> checked;
    s.push ("1");
    checked.insert("1");
    while (!s.isEmpty()) {
        auto curr = s.pop();
        if ( evaluate(curr) == goal) {
            printf("found! %s",curr);
            return ;
        }
        string s1 = curr + "*3";
        string s2 = curr + "/2";
        if (checked.find(s1) == checked.end()) {
            s.push(s1);
            checked.insert(s1);
        }
        if (checked.find(s2) == checked.end()) {
            s.push(s2);
            checked.insert(s2);
        }
    }
}
```

Version 0.2 using recursive

```
void DFS(int goal,int curr,unordered_set<int> &checked)
{
    if (curr == goal) {
        printf("found\n");
    } else {
        if (checked.find(curr * 3) == s.end()) {
            checked.insert(curr * 3);
            DFS(goal,curr * 3,s);
        }
        if (checked.find(curr / 2) == s.end()) {
            checked.insert(curr / 2);
            DFS(goal,curr / 2,s);
        }
    }
}
```

- We also change from state as a string into an integer which is the result
- Issue
 - Path is lost

Version BFS

```
void BFS(int goal) {
    queue q;    // queue of int
    hash h;    // hash of int
    q.push(1); hash.insert(1);
    while (S.isEmpty() = false) {
        curr = q.front(); q.pop();
        if (curr == goal) {
            printf("found!\n");
            return ;
        }
        if (!h.contain(curr * 3)) {
            q.push(curr * 3);
            h.insert(curr * 3);
        }
        if (!h.contain(curr / 2)) {
            q.push(curr / 2);
            h.insert(curr / 2);
        }
    }
}
```

- What's the difference?

Conclusion

- DFS
 - Stack-based
 - Use less space
 - Might loop forever if solution set is infinite
- BFS
 - Queue-based
 - Use large space
 - Found solution nearest to the root node in the search tree

Backtracking

Technique to reduce enumeration

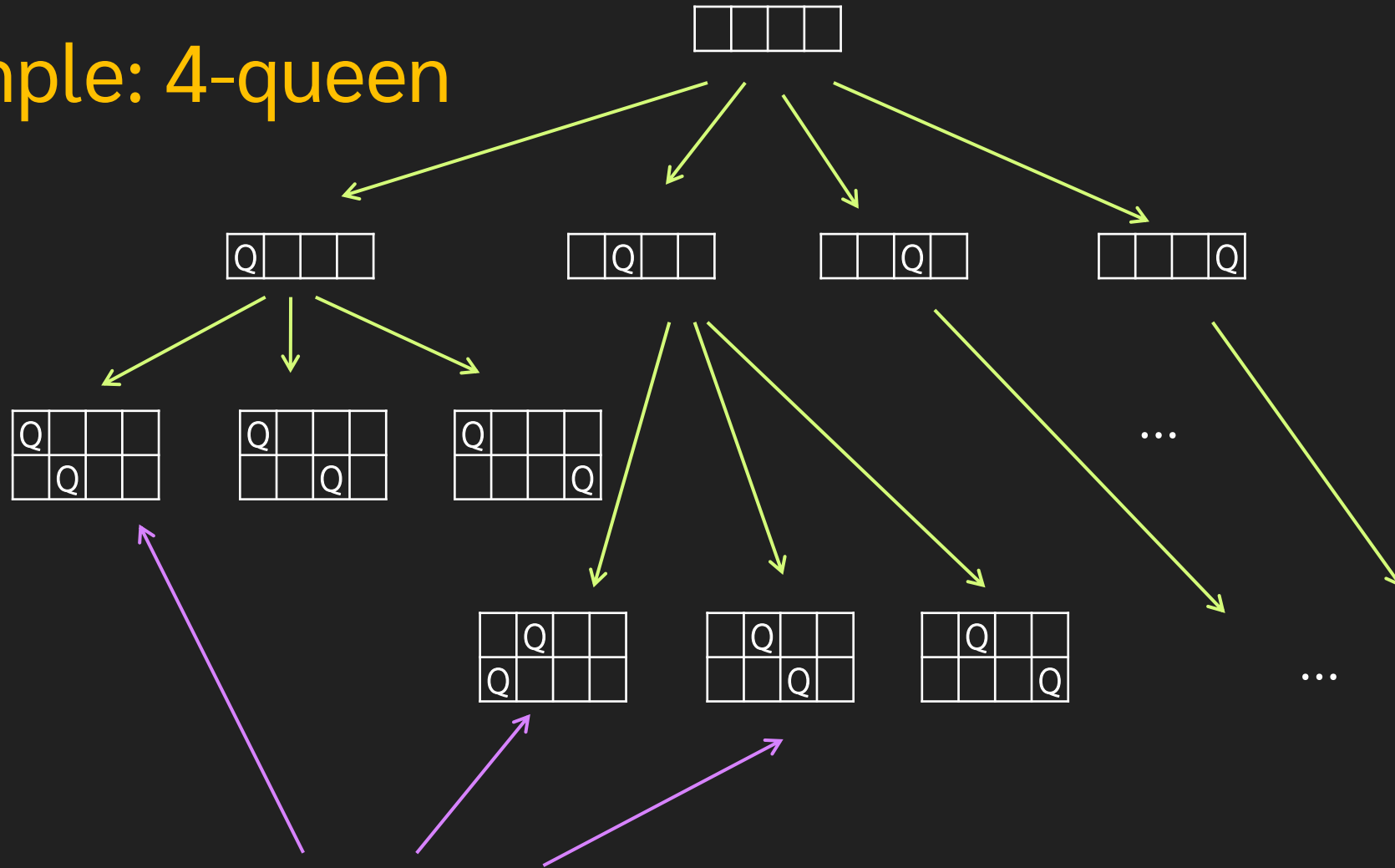
Main Idea

- We should not generate states that will never produce a solution.
- Backtracking using the information of a **partial state** (part of solution that have been generated) to determine whether the state can lead to a candidate solution that satisfies the constraints.
 - If it cannot, just stop.

Partial Solution

- Partial Solution
 - state that has more step, there will be more state directly generated from this state.
 - Partial state can also be a candidate solution (as in Triple and Half problem).
- Backtracking and B&B work with problem with partial solution by stopping a partial state from generating more state.
 - Stopping a state that, by using information from the partial part, we know that the partial state will not lead to a solution.
 - Effectively reduce the number of states and candidate solutions we must consider.

Example: 4-queen



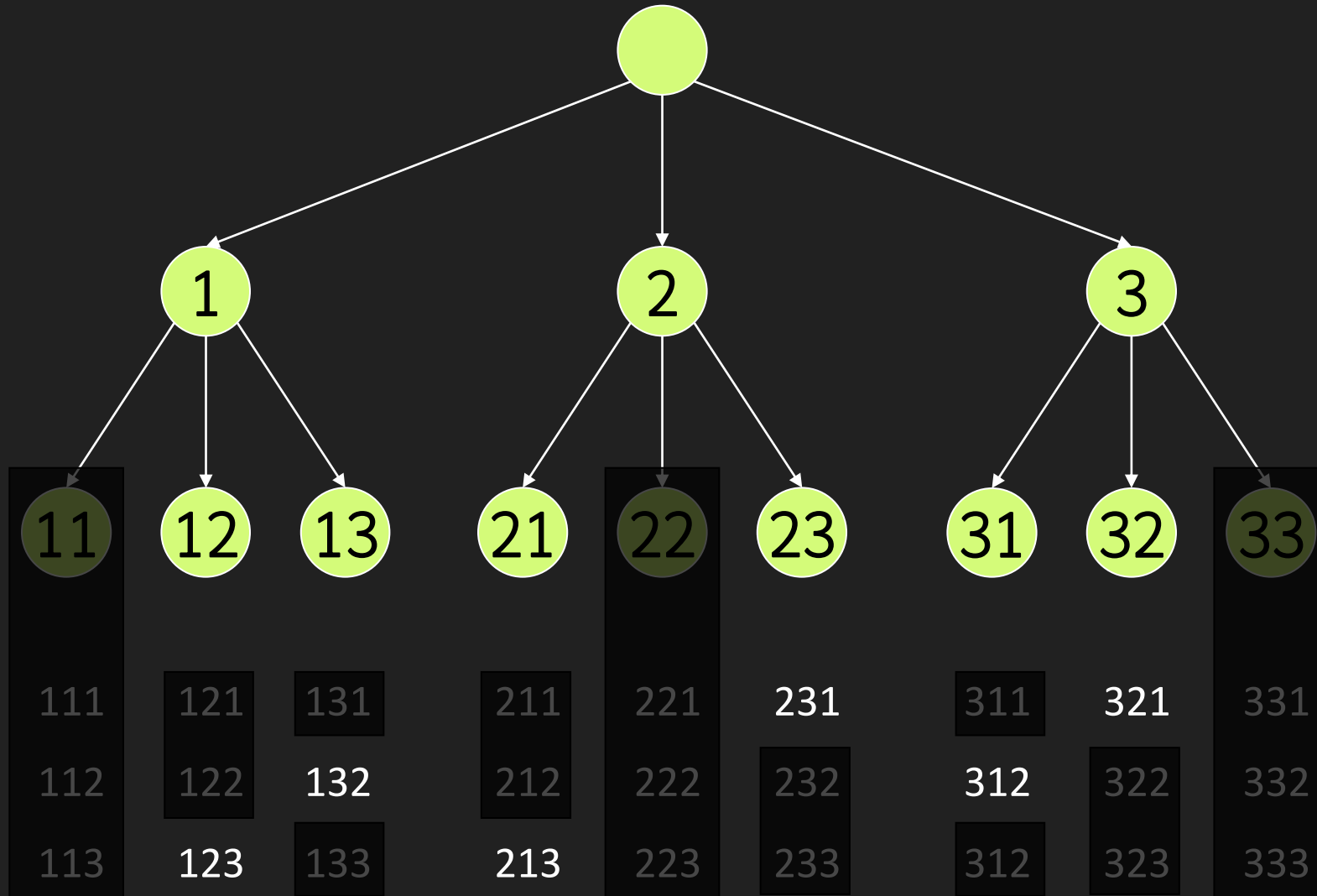
Should we proceed on
these state?

Example: Permutation by Combination

- Generate all permutations of $\{1..n\}$ by generating all combinations of n^n .
 - N steps, each step can be 1 to n.
 - At the candidate solution, check if the combination contains each number once.
- This is very slow because n^n is much larger than $n!$.

```
void permu(int step, vector<int> &a) {  
    if (step <= n) {  
        for (int i = 1; i <= n; i++) {  
            a[step] = i;  
            permu(step+1, a);  
        }  
    } else {  
        // check if a has each number once  
        if (check(a))  
            display(a)  
    }  
}
```

Permutation by Combination



```
void perm(int step,
          vector<int> &a,
          vector<bool> &used) {
    if (len < n) {
        for (int i = 1; i <= n; i++) {
            if (used[i] == false) {
                used[i] = true;
                sol[len] = i;
                perm(n, sol, len+1, used);
                used[i] = false;
            }
        }
    } else {
        display(a);
    }
}
```

Sum of Subset Problem

- Input:
 - Array $D[1..n]$ of positive integers.
 - A number K
- Output
 - A subset of D whose summation is K
- Example
 - $D = \{2, 5, 7, 1, 3, 8\}$
 - $K = 9$
 - Solution is $\{2, 7\}$ or $\{8, 1\}$ or $\{5, 3, 1\}$

Sum of Subset

- Search Space
 - Every possible subset of elements
 - Described by a sequence of selection bit
 - [1,0,1,0,0,0]
 - There are 2^n candidate solutions
- Generating all solutions
 - Step: choose either we select each item
- Checking each solution
 - Sum of the selected item
 - and test whether it equals to target

Sum of Subset by DFS, using recursive

```
void search(int step,int* sol) {  
    if (step < n) {  
        sol[step] = 0;  
        search(step + 1,sol);  
        sol[step] = 1;  
        search(step + 1,sol);  
    } else {  
        int sum = 0;  
        for (int i = 0;i < n;i++)  
            if (sol[i] == 1) sum += D[i];  
        if (sum == K) { printf("YES!\n"); }  
    }  
}
```

k = target
D = given array
n = size of array

Sum of Subset

- Sum the selected value along the recursive. A tad faster.

We don't really need to compute sum at the final step
Because **total** already do that for us

```
void search(int step,vector<int> &sol,int total) {  
    if (step < n) {  
  
        sol[step] = 0;  
        search(step + 1,sol,total);  
        sol[step] = 1;  
        search(step + 1,sol,total+D[i]);  
    } else {  
        if (total == K) { printf("YES!\n"); }  
    }  
}
```

Backtracking for Sum of Subset

- If summation of selected element is more than K , stop
 - Because no matter what we do with the remaining step, The summation will always more than K .
 - This backtracking is “stop when we select too much”

This line is
backtracking

```
void search(int step, vector<int> &sol, int total) {  
    if (step < n) {  
        if (total > K) return;  
        sol[step] = 0;  
        search(step + 1, sol, total);  
        sol[step] = 1;  
        search(step + 1, sol, total + D[i]);  
    } else {  
        if (total == K) { printf("YES!\n"); }  
    }  
}
```

Backtracking for Sum of Subset

- There is another backtracking.
- stop when we select too few
 - When elements in remaining steps is not enough for us to get to K , stop.
 - Try write this one yourself.

```
void search(int step,vector<int> &sol,int total) {  
    if (step < n) {  
        if (total > K) return;  
        // another backtracking here.  
        sol[step] = 0;  
        search(step + 1,sol,total);  
        sol[step] = 1;  
        search(step + 1,sol,total+D[i]);  
    } else {  
        if (total == K) { printf("YES!\n"); }  
    }  
}
```

Branch-and-Bound

Backtracking for COP

Branch and Bound

- B&B is backtracking for a **constraint optimization problem** (COP)
- B&B and backtracking both consider partial state and try to stop the partial state from generating more state
 - **Backtracking considers the known part** of the partial
 - E.g., backtracking for 8 Queen, for a partial state with 3 queen positions, we stop when any of these 3 queen can capture each other WITHOUT considering the remaining 5 queens that we have not placed yet
 - **B&B considers the unknown part** (step that we have not performed yet) by trying to determine best estimation of the unknown and decide weather we should search further down this step.
 - B&B uses a **bounding heuristic** to estimate the best possible outcome of the unknown part.
- B&B stops a partial state when its prediction is less than the current best score (from any evaluated candidate solution)

Constraint Optimization Problem (COP)

- A constraint satisfaction problem (CSP) with an objective function
- Goal of CSP is to find a solution that pass all the constrains
 - 8 queen: “no capturing pair” constraints
 - Subset sum: “summation equal to K” constraints
- For COP, we want to find a solution that pass all the constraints and have the best value of the objective function
 - Example: 0-1 Knapsack problem
 - Constraint: total weight of selected items does not exceed the capacity of our sack
 - Objective function: Maximize the summation of value of selected items
- B&B uses a bounding heuristic to estimate the value of the objective function of a partial state.

Bounding Heuristic

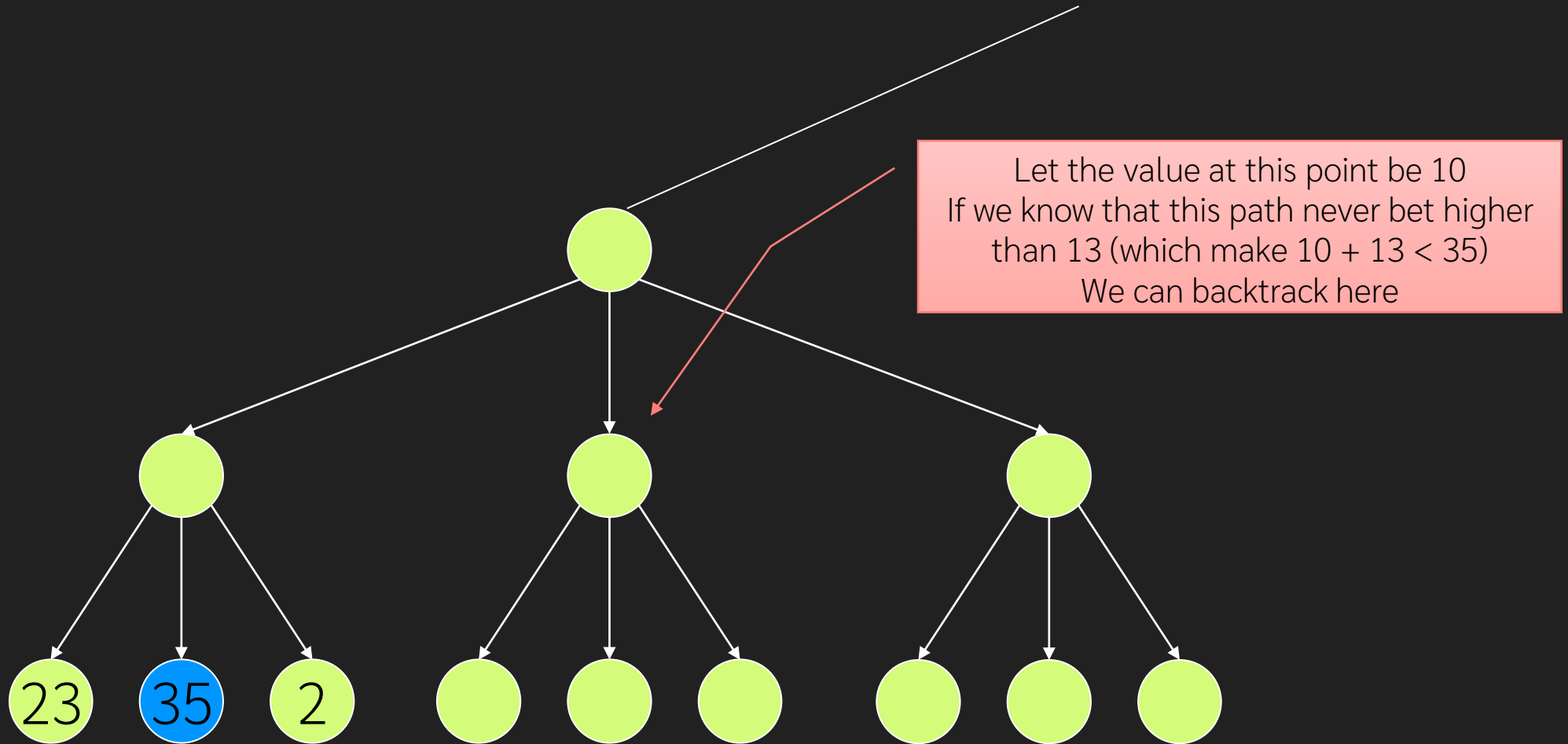
- Bounding heuristic is a function that take a partial state as an input and produce a prediction an objective function of the state.
- The goal of bounding heuristic is to predict the possible value of an objective function from incomplete information (partial state)
 - The bounding heuristic MUST NOT be underestimate
 - The bounding heuristic CAN BE overestimate
 - The bounding heuristic SHOULD BE fast, we trade of accuracy for time.
- With a good bounding heuristic, we can stop working on a partial state.

Example

- The problem is to find a maximum score over all candidate solutions.
- Let's assume that we have 5 steps (solution = [-,-,-,-,-])
- We have searched several candidate solutions already and the best one we have has a score of Z
- let assume that now we are at depth 3 where solution = [a,b,c,-,-]
 - The known part (a,b,c) gives a score of X
 - Our heuristic says that the unknown part (-,-) can give AT BEST score of Y
 - If $X+Y < Z$, there is no point in continuing the search, we should backtrack here

$X+Y$ is our prediction
of this partial state

Example

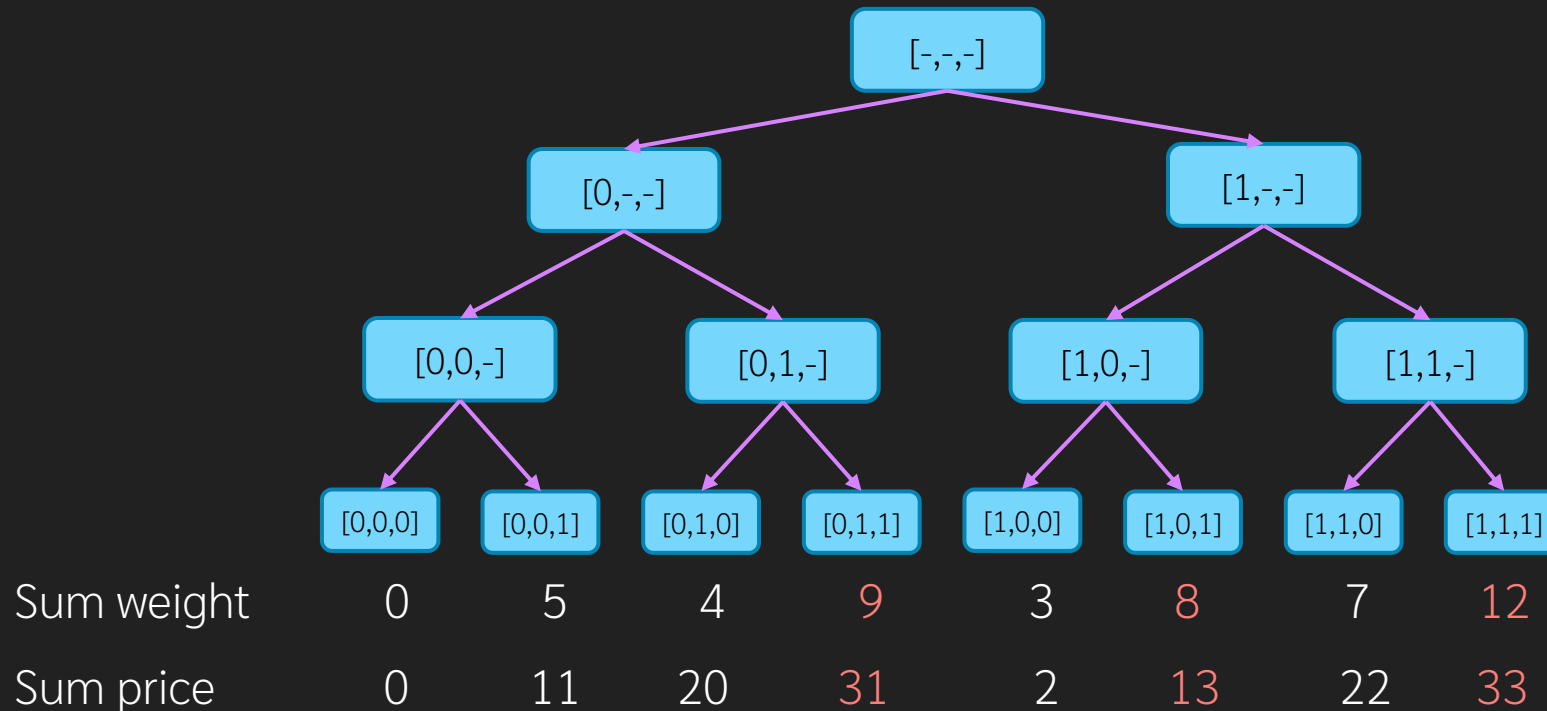


The Problem

- Task: There is a sack that has capacity W , we need to pack some items from a list into the sack such that the capacity is not exceed while the price of the item is maximum
- Input
 - A positive number W , the capacity of the sack
 - N , number of items
 - $w_1, w_2, \dots, w_n$, weights of each item
 - $p_1, p_2, \dots, p_n$, prices of each item
- Output
 - A subset S of $\{1, 2, 3, \dots, n\}$ such that
 - Weight not exceed $\sum_{i \in S} w_i < W$ (This is our constraint)
 - $\sum_{i \in S} p_i$ is maximum (This is our objective function)

Example of Instance and search tree

- $W = 7, N = 3, w = [3,4,5], p = [2, 20, 11]$
- Candidate solution set = power set of $\{1,2,3\}$
- Candidate solution = $A[1..n]$ where $a[i]$ indicates whether we select the i -th items

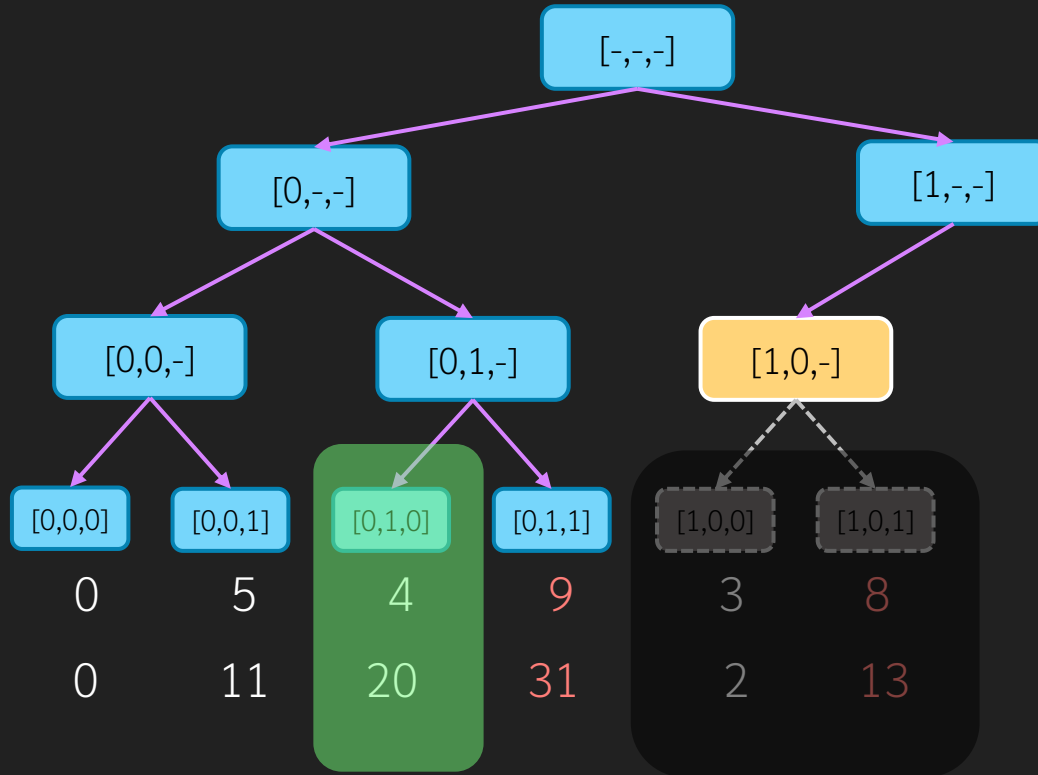


Example of Instance and search tree

$W = 7$
 $N = 3$
 $w = [3, 4, 5]$
 $p = [2, 20, 11]$

Sum weight

Sum price



Our best
solution
for now

- Let us consider the state [10-]
 - The state [010] and [011] have not been generated yet
 - Our best answer so far is [010] with sum price = 20
- The **known part** is [10] which means we take the first item but not the second, while the third is unknown
- On hand, we have sum weight = 3 and sum price = 2
- Let assume that **our bounding heuristic** predicts that, at [10-], the unknown part can give, **AT BEST**, additional price of 15, (a little bit overestimate)
- We can safely **STOP generating more children state of [10-], because**, AT BEST we will have our objective function value of $2 + 15 = 17$, which is still less than 20

Knapsack by Search

```
void knapsack(int step, vector<int> &sol) {  
    if (step < n) {  
        sol[step] = 0; knapsack(step + 1, sol);  
        sol[step] = 1; knapsack(step + 1, sol);  
    } else {  
        int sumP = 0;  
        int sumW = 0;  
        for (int i = 0; i < n; i++)  
            if (sol[i] == 1) {  
                sumP += p[i];  
                sumW += w[i];  
            }  
        if (sumP > max && sumW <= W) { max = sumP; }  
    }  
}
```

Knapsack by Search rev 2.0

```
void knapsack(int step, vector<int> &sol,int sumP,int sumW) {  
    if (step < n) {  
        sol[step] = 0;  
        knapsack(step + 1,sol,sumP,sumW);  
        sol[step] = 1;  
        knapsack(step + 1,sol,sumP + p[step],sumW + w[step]);  
    } else {  
        if (sumP > max && sumW <= W) { max = sumP; }  
    }  
}
```

- Same trick as before, calculate summation on the fly
- Sum of subset and knapsack is closely related

Knapsack with Backtracking

- Knapsack is like the sum of subset problem
- In SS, we need summation of element to be equal to K
- In KS, we need summation of weight to be less than W
- So, obviously, we should backtrack when weight sum of selected item is more than W

Knapsack with Backtracking

This line is
backtracking

```
void knapsack(int step, vector<int> &sol, int sumP, int sumW) {  
    if (sumW > W) return ;  
    if (step < n) {  
        sol[step] = 0;  
        knapsack(step + 1, sol, sumP, sumW);  
        sol[step] = 1;  
        knapsack(step + 1, sol, sumP + p[step], sumW + w[step]);  
    } else {  
        if (sumP > max && sumW <= W) { max = sumP; }  
    }  
}
```

Stop when **sumW** is more than **W**

Branch and Bound in Knapsack

- Let us consider a partial state of [xxxxx-----], where x indicates a known part of our partial while - is the unknown part.
- For the known part, we can calculate the known sum price.
- For the unknown part, we use a bounding heuristic that is the summation of the value of every item in the unknown part.
 - This is obviously NOT UNDERESTIMATE because, in reality, we might be able to choose only some of the unknown part, due to weight constraint.
 - We might be VERY overestimate, but that is OK.
 - Better heuristic exists.

Knapsack with Backtracking + B&B

tail[i] is the summation of price of item i, i+1, i+2,..., N

```
tail [n-1] = v[i];  
for (int i = n - 2; i >= 0; i--) tail[i] = tail[i+1] + p[i];
```

This line is backtracking

```
void knapsack(int step, int* sol, int sumP, int sumW) {  
    if (sumW > W) return ;  
    if (tail[step] + sumP < max) return ;  
    if (step < n) {  
        sol[step] = 0;  
        knapsack(step + 1, sol, sumP, sumW);  
        sol[step] = 1;  
        knapsack(step + 1, sol, sumP + p[step], sumW + w[step]);  
    } else {  
        if (sumP > max && sumW <= W) { max = sumP; }  
    }  
}
```

This line is Branch and Bound

- Stop when **sumW** is more than **W**
- Stop when remaining + **sumP** is less than current **max**

Knapsack with Backtracking + B&B rev 2.0

```
tail [n-1] = v[i];  
for (int i = n - 2; i >= 0; i--) tail[i] = tail[i+1] + p[i];
```

```
void knapsack(int step, int* sol, int sumP, int sumW) {  
    if (sumW > W) return ;  
    if (tail[step] + sumP < max) return ;  
    if (step < n) {  
        sol[step] = 1;  
        knapsack(step + 1, sol, sumP + p[step], sumW + w[step]);  
        sol[step] = 0;  
        knapsack(step + 1, sol, sumP, sumW);  
    } else {  
        if (sumP > max && sumW <= W) { max = sumP; }  
    }  
}
```

Why?



Does the search tree
differ?

Variation

- Rational Knapsack
 - Object is like a gold bar; we can cut it in to piece with the same value/weight
- Can be solved by greedy
- Sort object according to value/weight ratio
- Pick objects by that ratio
 - If object is larger than the remaining capacity, just divide it

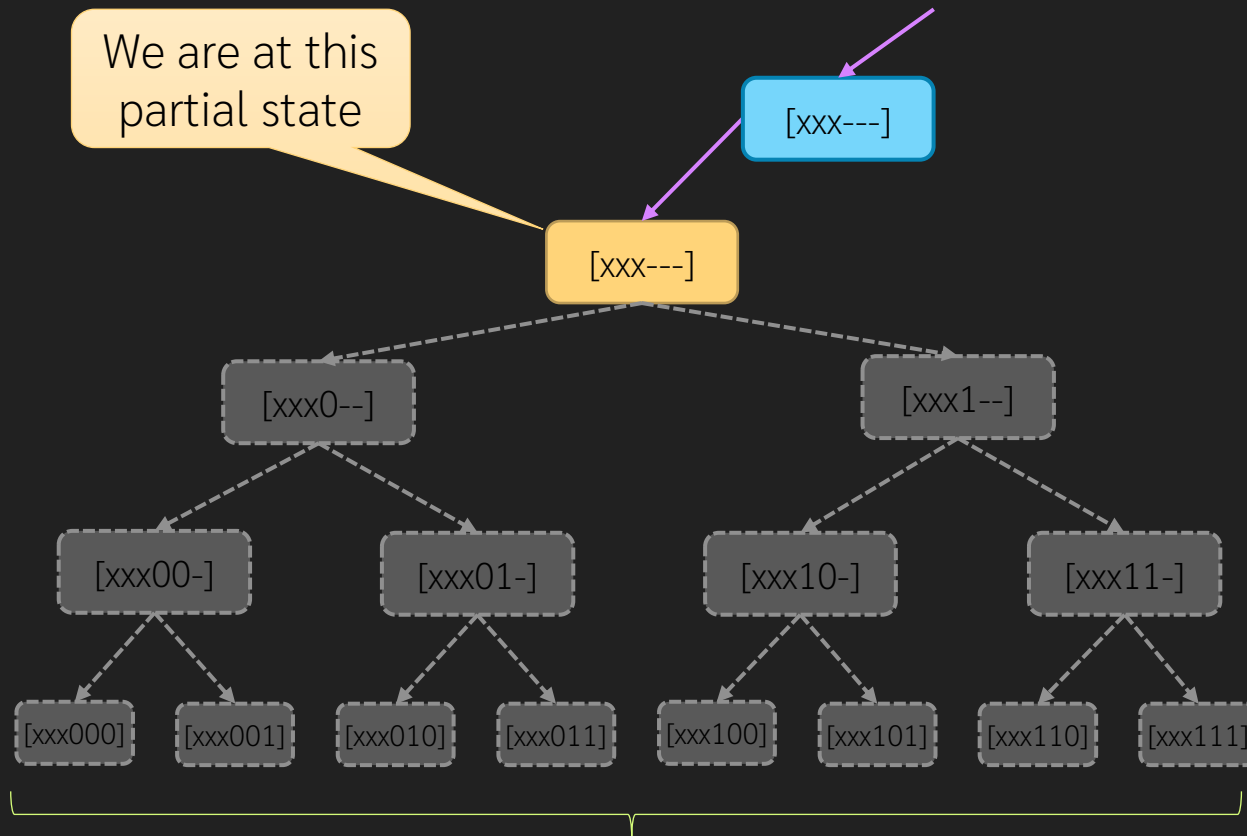
Another Example: Coin Change Problem

- Task
 - There are N types of coin denomination, e.g., (1 Baht, 2 Baht, 5 Baht, 10 Baht)
 - We want to pay exactly K Baht using minimal number of coins
 - For example, to pay 7B, the answer is to use 1 of 2B and 1 of 5B coin. While 2 of 1B with 1 of 5B is also 7 Baht, it is not the solution because the number of coin is higher
- Input:
 - K , the value to pay
 - $A[1..n]$ is the denomination of coins (Assume that $A[1]$ is always 1 so that we can always pay any value of K)
- Output
 - $C[1..n]$ where $C[i]$ is the number of $A[i]$ Baht coins
 - $\text{Sum}(C[i] * A[i])$ must be equal to K (our constraint)
 - $\text{Sum}(C[i])$ must be minimum (our objective function)
- Search Space
 - $C[i]$ can be any value from 0 to $K/A[i]$

Bounding Heuristic for Coin Change Problem

- Let us consider a partial state $C = [2, 3, -, -]$
 - Which means we now use 2 coins of $A[1]$, 3 coins of $A[2]$
 - Our current value is $2 * A[1] + 3 * A[2]$
 - So, we need $K - 2 * A[1] + 3 * A[2]$ Baht using $A[3]$ and $A[4]$
 - What is, AT BEST, the number of coins, using only $A[3]$ and $A[4]$ to get $K - 2 * A[1] + 3 * A[2]$
- This problem is a minimization problem
 - To overestimate (being optimistic) our heuristic function must give the value that is LESS than or equal to the actual objective function.

Bounding Heuristic Summary



- The bounding heuristic of [xxx---] tries to predict what is the best value of [xxx000], [xxx001], [xxx010], ..., [xxx111] without actually compute them.
- It must give something that is not worse than the best of the descendant candidate solutions of [xxx---].

Bounding Heuristic Summary

- Key
 - Never underestimate!!!
 - Future cost can be overestimate
 - Bounding heuristic is optimistic
- Maximization problem (finding highest value solution)
 - Bound must be higher than (or equal to) the real value (upper bound)
 - Good bound $\rightarrow$ lowest value that is $\geq$ the real value
- Minimization problem (finding lowest value solution)
 - Bound must be lower than (or equal to) the real value (lower bound)
 - Good bound $\rightarrow$ highest value that is $\leq$ the real value

Least Cost Search

a.k.a Best First Search

Least Cost Search

- For optimization problems, candidate solutions with high objective function allow B&B to prune the tree better
- Order of candidate solution is now important, while BFS and DFS allow us to change the order of candidate solution, we need a better way to aggressively guide it.
- We should try to **find good solution as fast** as possible (so that B&B can benefit from this one)
- Search toward **promising** solution

Guided search

Least Cost Search

- Instead of stack or queue, we use priority queue in search framework
- Key value in PQ is calculated from the partial solution using the bounding heuristic function
 - The same one in the B&B
- It's called Best First Search
 - Guided by the value of partial solution
 - Suffers similar problem as BFS (since we need to maintain the queue)

LC-Search with Branch and Bound

- Use **bounding heuristic** for value in PQ
- The data stored in the PQ is ordered by our prediction value of the partial state, usually in the form of $X+Y$
 - X = Value of the known part of the partial solution
 - Y = Value from the unknown part of the partial solution
- I.e., use the **bounding heuristic** to guiding the search
 - For maximization problem use **upper bound of total value**
 - For minimization problem use **lower bound of total value**
- Can guarantee optimality of the first solution

15 Puzzle Problem

- Given a puzzle board
- Objective is to move pieces around so that the board be like the figure on the right



15 Puzzle Problem

- Input
 - A board (solvable)
 - $b[1..4][1..4]$ where $b[i][j]$ indicates the piece at row i col j
 - $b[i][j] = 0$ means the empty slot
- Output
 - A sequence of valid move that solve the puzzle (our constraint)
 - Using minimal moves (our objective function)
- This is Minimization problem

15 Puzzle Problem

- Search Space
 - Every possible sequence of movement
 - Described by a sequence of move of the “empty” piece
 - [U,D,D,L,U,R,...]
 - Don't know the limit on the length of sequence
- Checking each solution
 - Simulate the move, see whether it leads to solution

1		3	4
6	2	11	10
5	8	7	9
14	12	15	13



Move the space down

1	2	3	4
6		11	10
5	8	7	9
14	12	15	13



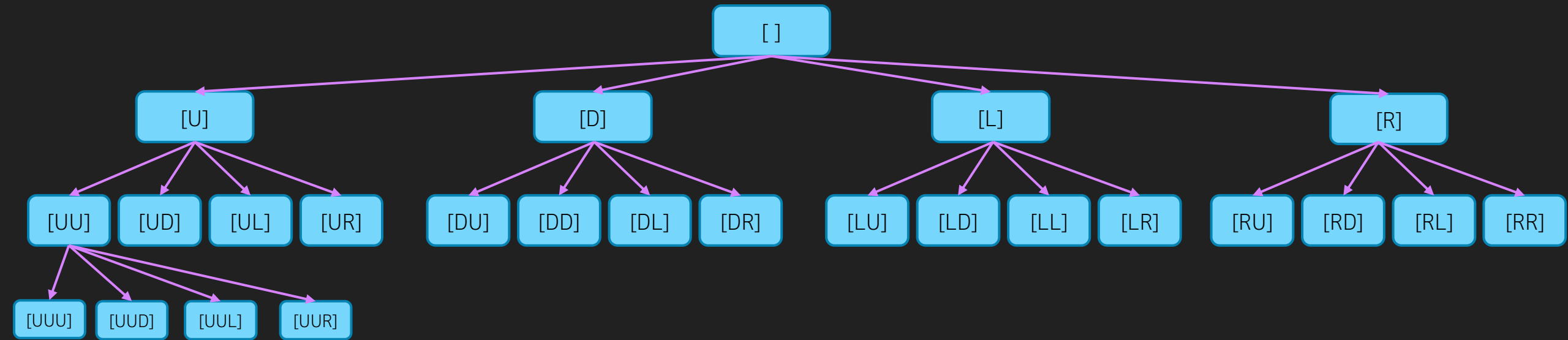
Move the space right

1	2	3	4
6	11		10
5	8	7	9
14	12	15	13

This state is [Down,
Right] (or [U,D])

Similarity to the * 3 / 2 problem

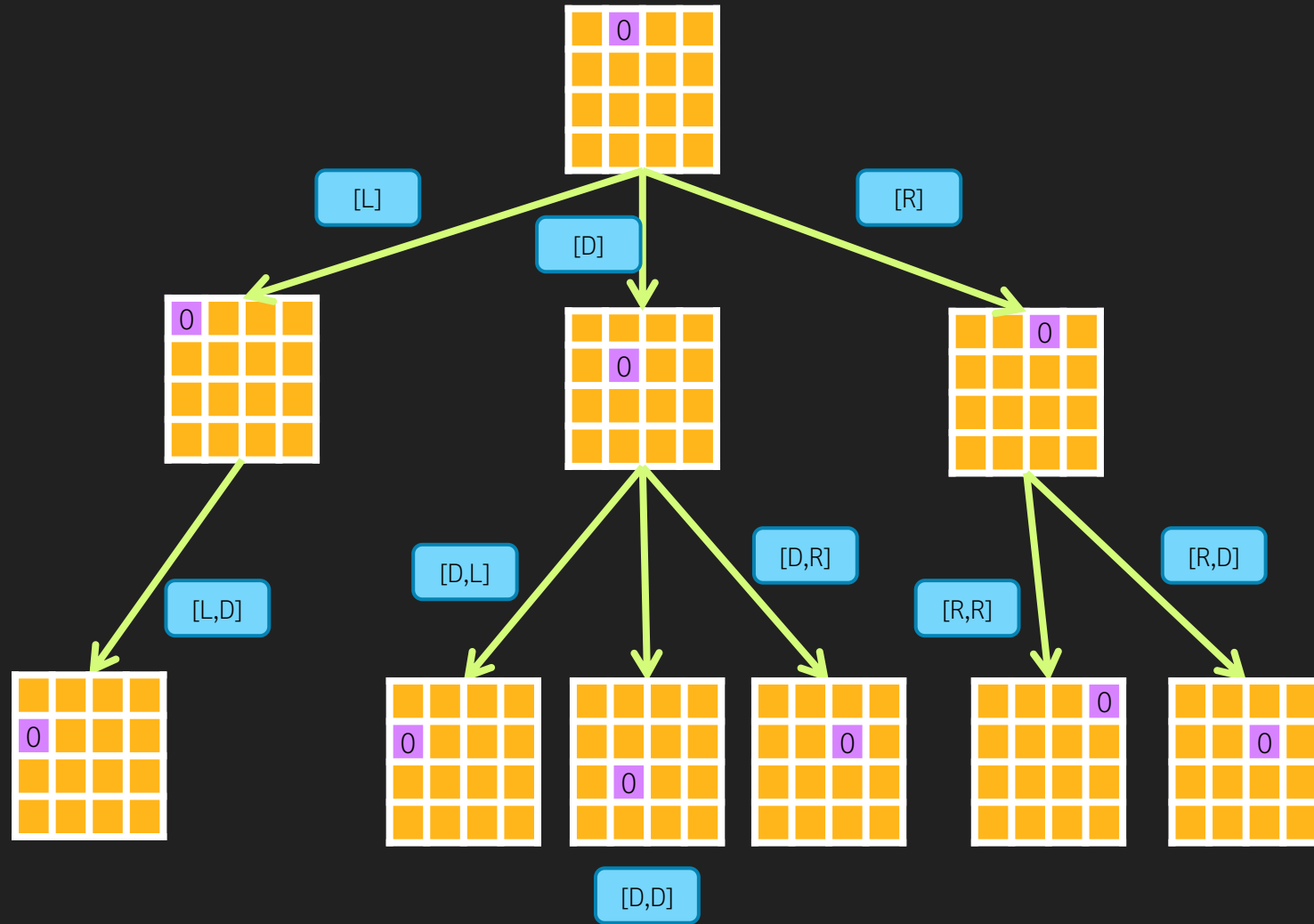
- There are different moves that gives the same board
 - E.g., [L,R] and [L,R,L,R]
- 15 Puzzle is infinite search space
- Every partial state is actually a candidate solution



Backtracking

- Check boards that we have generated
 - This will limit reverse move (e.g., if the last step is Up, we cannot go Down)
Hence, each state will have at most 3 possible move (except for the initial state)
- Cannot move the space “outside” the board
 - This will further limit the move. Only the center 4 cell that have all possible move.

Search Tree with Backtracking



Misplace = 12

1		3	4
6	2	11	10
5	8	7	9
14	12	15	13

Bounding Heuristic for 15-Puzzle

- Number of misplaced piece
- Obviously is a lower bound because each misplaced need at least one move to go to correct place

left

right

down

Misplace = 11
Moved = 1

	1	3	4
6	2	11	10
5	8	7	9
14	12	15	13

1	3		4
6	2	11	10
5	8	7	9
14	12	15	13

1	2	3	4
6		11	10
5	8	7	9
14	12	15	13

Misplace = 13
Moved = 1

Misplace = 13
Moved = 1

11 + 2

11 + 2

10 + 2

1	2	3	4
	6	11	10
5	8	7	9
14	12	15	13

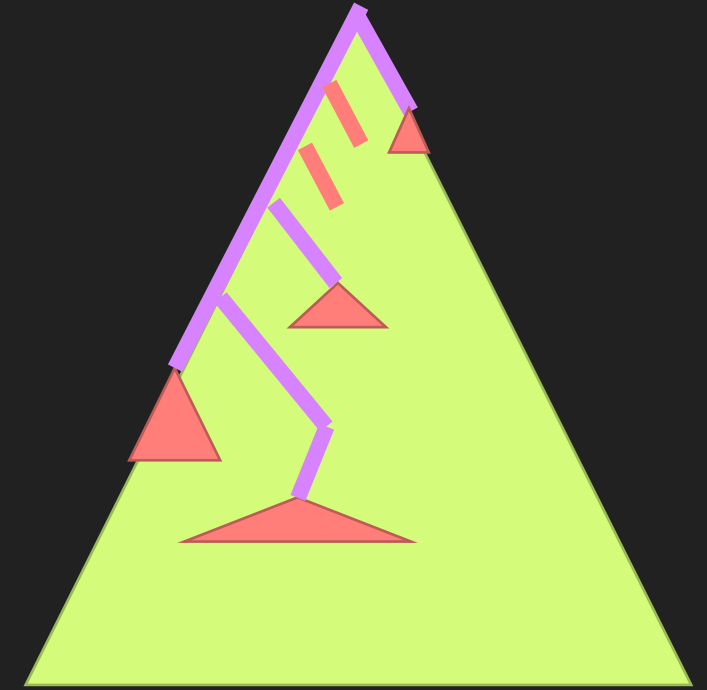
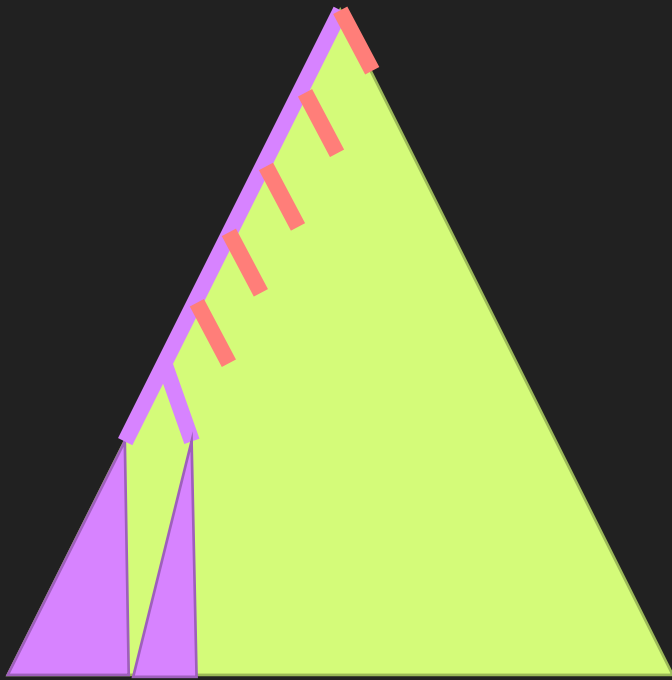
1	2	3	4
6	11		10
5	8	7	9
14	12	15	13

1	2	3	4
6	8	11	10
5		7	9
14	12	15	13

Comparing Search Strategy

- Both DFS, BFS and Best First Search works in the same way
- There are 3 types of state:
 - Visited: state that we have already considered, and all its direct children is generated.
 - In-progress: state inside the storage, waiting to be processed
 - Unseen: state that we have not seen yet. It might be generated in the future, or it might be discarded by backtracking or branch and bound
- All search starts with the initial states as in-progress, no state as visited and all other states as unseen. As the algorithm proceed, unseen state is either discarded or changed into in-progress which will eventually become visited.
- DFS, BFS and Best First Search have different ordering of state

DFS, BFS, Best First Search



visited

in-progress

unseen

Another Bounding Heuristic

- L-1 Distance of misplace piece
- For example, consider piece #12
 - It is at row 4 col 2
 - It should be at row 3 col 4
 - Distance = 1 row 2 col = $1 + 2 = 3$
- Obviously, it is a lower bound as well
 - Better lower bound (closer to the actual # of moves)

current			
1		3	4
6	2	11	10
5	8	7	9
14	12	15	13

goal			
1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	